

JavaScript mid-term exam (2022-23-1)

Due No Due Date Points None

Important information

- You have **180 minutes** to complete the exam. Filling in the included `README.eng.md` template, reading the tasks, downloading the materials, compressing and uploading your solution is **included** in the time limit.
- Solutions must be uploaded to ELTE Canvas. **The system closes at 17:00 (Hungarian time) and there will be no opportunity to submit your solution after the aforementioned time.**
- You may use **any kind of material** (including documentation, lectures, class exercises, cheat sheets, etc) during the exam, **except human help** (synchronous, asynchronous, chat, forums, etc)! By filling in the `README.eng.md` file, you accept the possible consequences of cheating.
- The tasks are independent and can be solved **in any order**.
- To begin your work you should first [download the starting package we have prepared](#). Each task has its own directory in the package. All tasks are somewhat prepared. Generally, you only have to edit the provided `.js` files, but feel free to divide up your solution into multiple `.js` files, or even edit the HTML if needed.
- The starting package contains a template called `README.eng.md` where you should fill in your name and Neptun code (the `<>` characters must be removed). **Submissions without a properly filled `README.eng.md` file will not be graded!**
- Each directory contains a file named `TASKS.md`. Replace the space in `[ ]` with an `x` where you have (even partially) succeeded to complete the task. This will make us easier to find your solution when we grade your exam.

Task 1: Tic-tac-toe (1-tictactoe, 8 pts)

The constant `game` defined in file `index.js` contains the current state in a game of Tic-tac-toe. Solve the following tasks using this constant. _Technical help: we recommend the use of array functions. You can use `split` (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/split) to create an array from a string._

- a. (2 pts) Output whether all rows have the same length to the element with ID `task1`. (*Does the length of each row match the length of the first row?*)
- b. (2 pts) Output whether the first row only contains X and O characters to the element with ID `task2`.
- c. (2 pts) Output the number of X and O characters to the element with ID `task3`.
- d. (2 pts) Output the index of any row that contains three consecutive Xs or three consecutive Os to the element with ID `task4`. (You may assume that at least one such row exists.)

TIC-TAC-TOE

Minden sor ugyanolyan hosszú? (true/false)

Do all rows have the same length? (true/false)

true

Az első sorban csak X vagy O karakter van? (true/false)

Does the first row only contain X and O characters? (true/false)

true

X és O karakterek száma:

Number of X and O characters:

X / O = 3 / 7

Egy olyan sor indexe, amelyben három O vagy három X van egymás mellett:

Index of any row that contains three consecutive Xs or three consecutive Os:

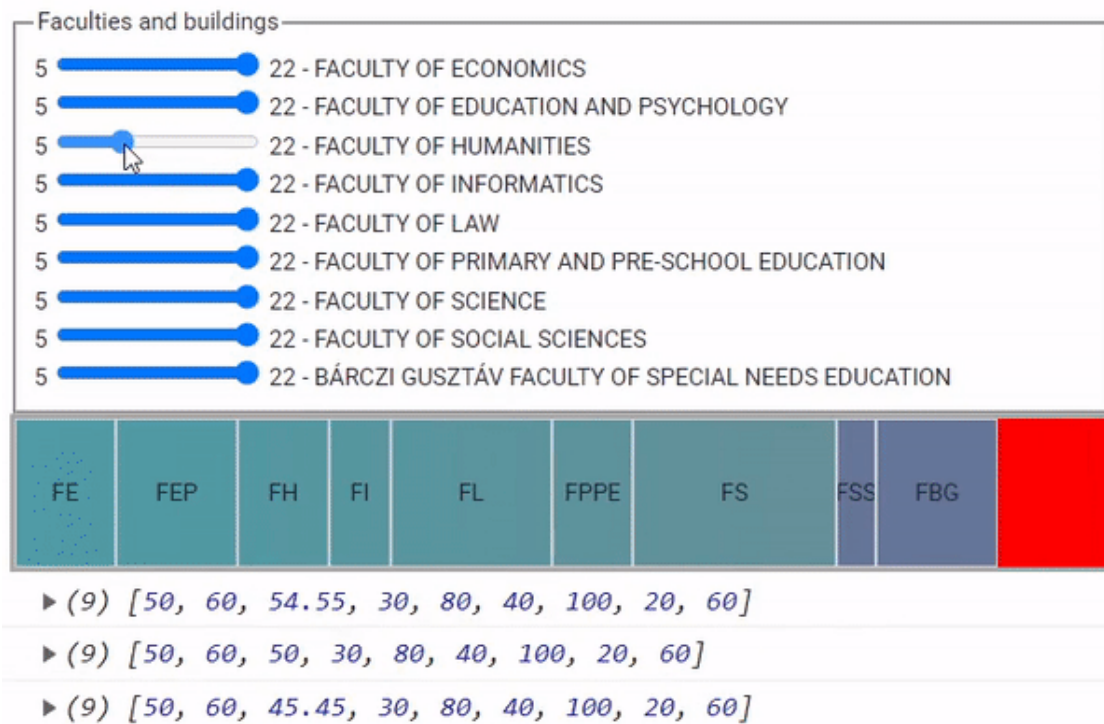
2

Task 2: Energy savings (2-savings, 8 pts)

Due to skyrocketing energy prices, it is particularly important for organizations with large buildings (such as ELTE) to save energy wherever it's possible. This is most easily achieved by keeping the temperature low. Help the University save 25% of the bill by adjusting the temperature of each building. The temperatures can be adjusted between 5 and 22 degrees Celsius using sliders. Last year's energy consumption of the buildings are given in the `data-consumption` attribute of the input fields.

- a. (2 pts) Calculate last year's total energy consumption (`M`) and output it to the console. For this, select all input fields, read the value of `data-consumption` attribute of each input as a number and calculate the sum.
- b. (2 pts) Calculate the current consumption (`ci`) for each input and log the values to the console. Read the current value (`value`) of the input, divide it by the maximum value (`max` attribute of the input) and multiply it by `data-consumption`. (`ci=(value/max)*consumption`) Note: the value of `ci` matches the value of `data-consumption` when the page loads since every slider is set to its maximum value.
- c. (2 pts) Every input has a corresponding `label` element on chart under the form. The label's `for` attribute matches the `id` attribute of the input it belongs to. Set the `label` element's width style property to `ci / M * 100` percent (%) for each input field (e.g. `'65%'`).
- d. (2 pts) Refresh the chart when the value of any input field is changed (event type: `input`). (Perform the previous three subtasks again.) Technical help: it is sufficient to listen for `input` events at the level of the form and recalculate the values.

ENERGY SAVINGS IN ELTE



Task 3: Parcel delivery (3-parcel, 8 pts)

Packages are delivered to remote locations by aircraft. Landing and take-off consumes a lot of fuel, but a skilled pilot can deliver a package without landing. Make a game using canvas where the first click starts the plane and a second click releases the parcel. Show your skill and deliver to the house!

The `index.js` file already contains the necessary basic structure for the game:

- the game loop (`next` - [as seen on the lecture](http://webprogramozas.inf.elte.hu/webprog/lectures/06/index.eng.html#/animation-loop-1))
- the functions needed to modify and display the game state (`update` and `render`)
- the necessary objects to store the game state (`plane`, `parcel` and `house`)

Tasks:

- (1 pt) Clear the canvas inside the `render` function and draw 3 rectangles to the position of the plane, the parcel (package / box) and the house.
- (1 pt) Replace the rectangles with the preloaded images.
- (2 pts) When the user clicks, start moving the plane: set its horizontal velocity (left to right speed, `vx`, e.g. 200px/s) and make it change position.
- (1 pt) Make the parcel move with the plane.
- (1 pt) When the user clicks again, drop the parcel. Give it a vertical acceleration (`ay`, e.g. 300 px/s²) to make it fall faster and faster (gain more downward speed continuously). You may use the variable `gameState` to track the current state of the game.
- (1 pt) Check whether the parcel touches the house on every position change (frame). If it does, show the text "Delivered" and stop the fall of the parcel.

- g. (1 pt) If the parcel reaches the bottom of the canvas (did not reach the house), show the text "Missed" on the canvas and stop the fall of the parcel.

PARCEL DELIVERY



Task 4: Memory game (4-memory, 8 pts)

Create a game where you have to remember the order of flashing circles and then play them back from memory.

- a. (2 pts) By moving the slider responsible for the number of circles, draw the amount of circles corresponding to the value of the slider in the element with the ID `container`. You can find an example of what a circle looks like in `index.html`.
- b. (1 pt) When the "Start" button is pressed, fill the `series` array with 7 random integers ranging from 1 to the number of circles (inclusive). Output the `series` array to the console.
- c. (1 pt) Also when the "Start" button is pressed, flash the first circle. Use the utility function `toggleHighlight` with the DOM element of the given circle as parameter. This function applies an animation to the selected element, then removes the animation as well.
- d. (1 pt) If you are able to flash a circle then iterate over the generated `series` array and flash the circles which correspond to the stored elements. If we used a simple loop, all the circles would flash at the same time. Let's iterate over the array with 1 second delay. Use the idea from the lecture: rewrite the for loop as a while loop, then transform the loop into recursion, finally replace the recursive call with a timer.

```
function foo() {
  for (let i = 0; i < 5; i++) {
    // something
  }
}
```

```
function foo(i = 0) {
  while (i < 5) {
    // something
    i = i + 1
  }
}

function foo(i = 0) {
  if (i < 5) {
    // something
    foo(i + 1)
  }
}

function foo(i = 0) {
  if (i < 5) {
    // something
    setTimeout(() => foo(i + 1), 1000)
  }
}
```

- e. (1 pt) When a circle is clicked, read the circle's number and insert it into the `solution` array. Check whether the length of the `solution` array has reached the length of `series`. If the length is equal, then check whether the two arrays are the same. Output the result to the console.
- f. (1 pt) Write "Flashing circles..." while the circles are flashing and "Now, your turn..." when the flashing is over into the element with the ID `output`. Use the same element at the end of the game to display either "Nice job!" or "Failed!" depending on the result.
- g. (1 pt) Use a state variable (`canGuess`) to ignore the player's input if the flashing hasn't finished yet.

MEMORY GAME

Number of circles:



Now, your turn...